AI HACKATHON 2026 · DIU CPC × UPAY · TRACK 05: MERCHANT & AGENT INTELLIGENCE

FloatWise

AI agent-liquidity forecaster for upay

Project report: the problem, the solution, the architecture, the technical workflow, the execution and the results.

Model version **fw-c078f4bb** · Test period 1 January – 30 June 2026 · Demo date 15 March 2026

Repository: github.com/redwanhasan980/DIU_AI_Hackathon

All data in this project is synthetic. Every number in this report is produced by code and read from `artifacts/*.json`.

Contents

1. Executive summary 3
Headline results 3
2. The problem 4
2.1 Context 4
2.2 Who loses on a failed cash-out 4
2.3 How big and how seasonal 4
2.4 Why today's rule of thumb fails 5
2.5 Problem statement 5
3. The solution 6
3.1 What FloatWise does 6
3.2 Users 6
3.3 A day with FloatWise 6
3.4 Worked example: AG00471, five days before Eid 6
4. System architecture 8
4.1 Layers 8
4.2 Technology stack 8
4.3 Data contracts 9
4.4 Precomputation 9
5. Synthetic data 10
5.1 Scale and split 10
5.2 Generation model 10
5.3 Agent archetypes 10
5.4 Alignment with upay 11
5.5 Data quality checks 11
6. Forecasting model 12
6.1 Targets and models 12
6.2 Features (known at the forecast origin) 12
6.3 Baselines and acceptance 12
7. Decision engine 14
7.1 Stockout probability 14
7.2 Recommendation 14
7.3 DSO allocation 14
7.4 Business values 14
8. Explainability and the language model 15
9. API and user interface 16
9.1 REST API (FastAPI, prefix /api/v1) 16
9.2 Screens 16
10. Business impact 19
10.1 Unit economics per 1,000 agents per month 20
10.2 At upay's network scale 20
10.3 One control: the service level 20
10.4 Fairness 21
11. Design choices made on held-out data 22
12. Responsible AI and security 23
13. Execution: how it was built and how to run it 24
13.1 Delivery phases 24
13.2 Pipeline 24
13.3 Run it (about 6 minutes) 24
13.4 Quality assurance 24
14. Limitations and next steps 26
14.1 What FloatWise does not fix yet 26
14.2 Path to a pilot 26
Appendix A. Glossary 27
Appendix B. Evidence index 28

1. Executive summary

Mobile financial services (MFS) agents in Bangladesh run out of **cash** (so they cannot pay cash-outs) or **e-float** (so they cannot accept cash-ins) on exactly the days when customers need them most: before Eid, on garment salary days, during remittance surges and on government allowance (G2P) days. The customer hears “cash nai, pore ashen” (“no cash, come back later”). upay loses the fee, the agent loses commission and the customer may switch wallets.

FloatWise forecasts every agent's cash-out and cash-in for the next 7 days, turns the forecast into a stockout risk and an exact top-up amount, plans the field officers' (DSOs') visits within their limits, explains each recommendation in Bangla or English, and leaves every decision to a person. A six-month backtest against today's rule of thumb measures the impact.

−47.5%
stockout agent-days

−53.7%
upay fee revenue lost

−53%
forecast error vs same day last week

Tk 9.3 Cr
upay fees recovered per month at network scale

Headline results

Result	Value	Source
Stockout agent-days, 500 agents, Jan–Jun 2026	6,973 → 3,664 (−47.5%)	simulation_default.json
Failed cash-out value	Tk 52.2 crore → Tk 27.8 crore (−46.7%)	simulation_default.json
upay fee revenue lost	Tk 33.81 lakh → Tk 15.64 lakh (−53.7%)	simulation_default.json
Cost: average cash held / DSO trips	+2.3% / +4.1%	simulation_default.json
Cash-out forecast error (WAPE), overall	18.7% vs 39.8% for same day last week	metrics.json
Cash-out forecast error on Eid and event days	21.2% vs 58.9% (−64%)	metrics.json
At upay's ~1,54,000 agents, per month	Tk 9.3 crore fee, Tk 6.4 crore commission, 1,68,938 stockout-days avoided, 2,69,962 customers kept	simulation_default.json
Fairness	Every area type and both rural and urban improve on forecast and on stockouts	fairness.json
Quality	89 automated tests, 21-step browser walk-through, CI on Linux and Windows, deterministic rebuild	repo

> **Design principle.** Machine learning predicts demand. Deterministic, tested code decides amounts. The language model only explains, behind a numeric guardrail. A person approves every action; no money ever moves automatically.

2. The problem

2.1 Context

upay serves customers through a network of roughly 1,54,000 retail agents (end of 2023). Each agent holds a fixed total float split between physical **cash** in the drawer and **e-float** in the agent wallet. A cash-out converts the agent's cash into e-float; a cash-in does the opposite. When either side reaches zero, one type of transaction stops.

Failure mode	What happens	Typical trigger
Cash stockout	Agent cannot pay a cash-out; customer is turned away	Pre-Eid remittance arrivals, garment salary days, G2P allowance days
E-float stockout	Agent cannot accept a cash-in; sender goes elsewhere	Urban pre-Eid rush to send money home, market days

2.2 Who loses on a failed cash-out

- **The customer** cannot withdraw salary or remittance on the day it is needed and may move to another wallet.
- **The agent** loses the 0.41% commission on every failed cash-in and cash-out, on the busiest days of the year.
- **upay** loses its cash-out fee (1.0–1.4% by wallet type) net of the 0.51% channel commission, and risks losing the customer.
- **The distributor and DSO** lose 0.10% distributor commission and spend the day firefighting with unplanned trips.

In the six-month backtest of 500 synthetic agents, today's rule of thumb failed **2,15,101 cash-outs worth Tk 52.2 crore**.

2.3 How big and how seasonal

Under today's rule, **30%** of agent-days stock out in the week before Eid-ul-Fitr, against **1.6%** on a normal day. The pain is concentrated in a few predictable windows, which is exactly what a forecaster can exploit.

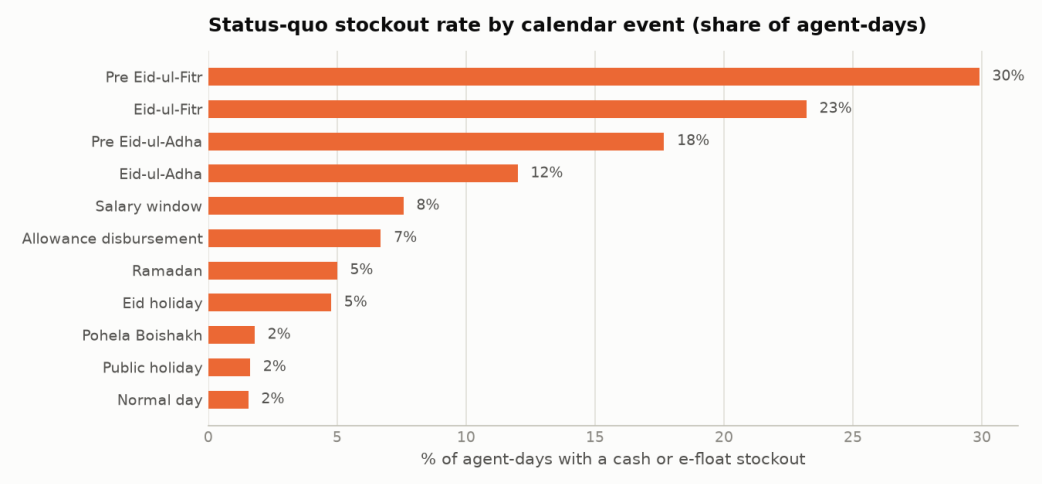


Figure 1. *Stockout rate under today's rule by type of day (synthetic network, full dataset).*

2.4 Why today's rule of thumb fails

Agents and DSOs top up to about 1.5× the last 7 days' average when the balance falls below 0.6× of that target. An average only sees the past. Eid moves about 11 days earlier every year, and salary and allowance days follow the calendar, not the average. For the demo agent the average keeps climbing during the pre-Eid rush and peaks on Eid day, after demand has already fallen by about three-quarters.

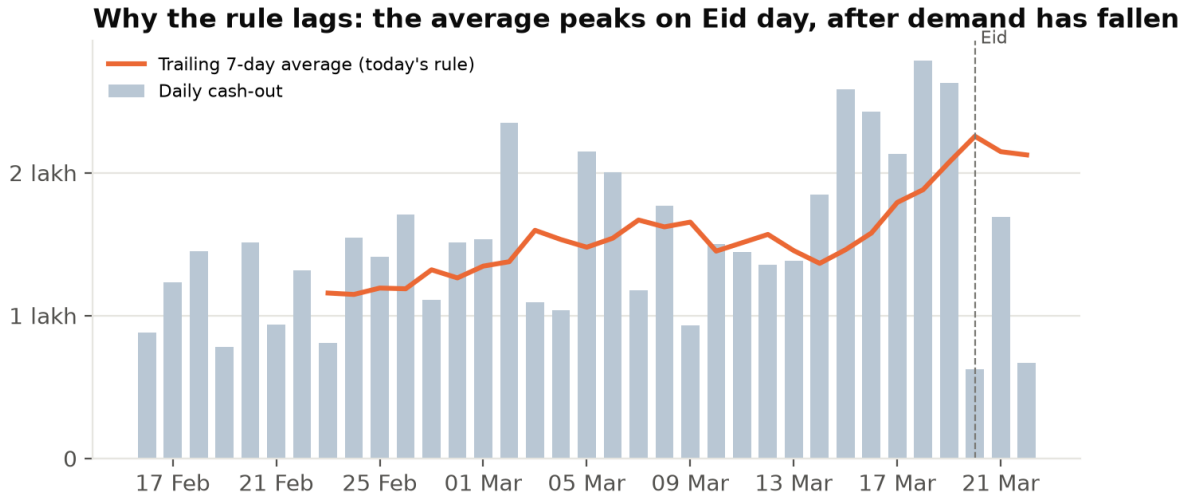


Figure 2. *Demo agent AG00471: daily cash-out against the trailing 7-day average used by today's rule.*

2.5 Problem statement

> **For** upay operations, distributors, DSOs and agents, **who** cannot see tomorrow's cash and e-float needs, FloatWise **is** an AI liquidity forecaster **that** predicts each agent's 1–7-day demand, sizes the top-up and plans DSO visits, **unlike** a backward-looking 7-day average, **because** it learns the Bangladesh calendar (Eid, salary, remittance, allowance days) and reports its uncertainty.

3. The solution

3.1 What FloatWise does

1. **Forecast.** Every agent's cash-out and cash-in for the next 1–7 days as quantiles (P10, P50, P90).
2. **Assess risk.** Convert the forecast into the chance of running out of cash or e-float tomorrow and over the next 7 days.
3. **Recommend.** TOP_UP_CASH, TOP_UP_EFLOAT, NO_ACTION or NEEDS_EXTRA_FLOAT, with an exact amount rounded up to Tk 1,000 and reason codes.
4. **Plan.** Rank DSO visits by taka at risk within the visit limit (10) and the cash bag (Tk 10 lakh); unvisited agents get the advice in the app.
5. **Explain and approve.** SHAP drivers plus a Bangla or English explanation; a person approves, modifies or rejects.
6. **Prove.** Backtest FloatWise against today's rule on six months of true demand, including Eid-ul-Fitr and Eid-ul-Adha 2026.

3.2 Users

Persona	Needs	FloatWise screen
Operations lead (upay / distributor)	Where is the network at risk, how much cash and e-float to move, is it working?	Network map, Impact, Model & fairness
DSO (field officer)	Which agents to visit today with a limited bag	DSO plan
Agent	How much cash to keep tomorrow and why, in Bangla	Agent mobile card
Risk / compliance	Is it fair, explainable and human-controlled?	Model & fairness, Decisions log

3.3 A day with FloatWise



Figure 3. *The daily operating loop.*

3.4 Worked example: AG00471, five days before Eid

Rahmania Bhandar (AG00471) is a rural remittance shop in Beanibazar, Sylhet. At close on 15 March 2026 it holds **Tk 18,501** in cash. FloatWise forecasts **Tk 2,60,567** of cash-out for 16 March (P10 Tk 2,14,385 – P90 Tk 3,03,469), a 100% chance of running out. It recommends **TOP_UP_CASH Tk 2,66,000 before opening**, limited by the agent's total float of Tk 2,88,000. The main SHAP drivers are days to Eid (+Tk 95,410) and Ramadan (+Tk 13,725). True demand fell inside the P10–P90 range on 6 of the 7 forecast days.

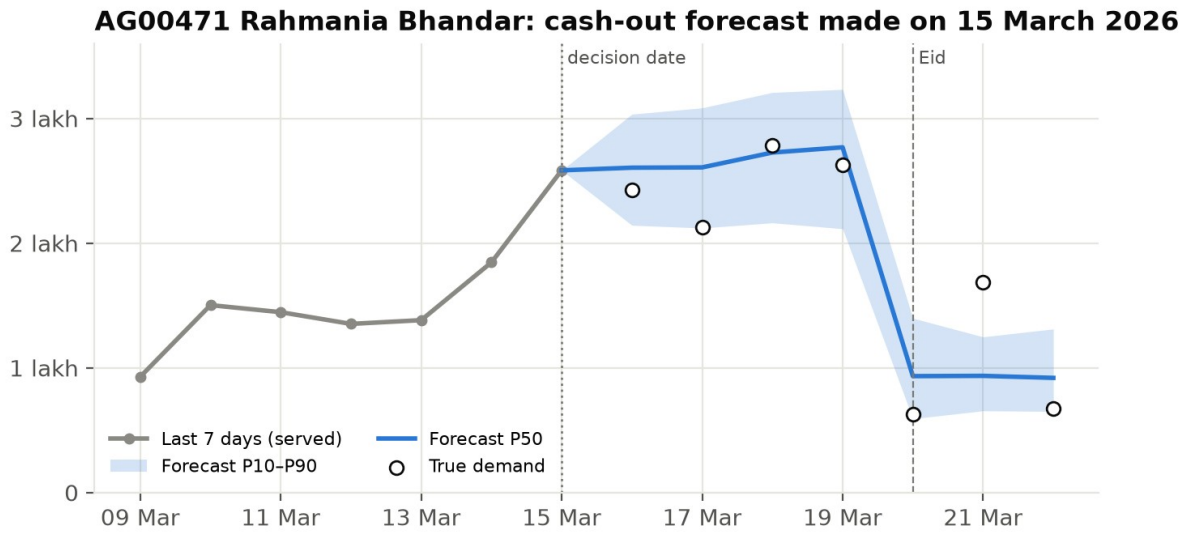


Figure 4. *Forecast made on 15 March against what really happened.*

4. System architecture

FloatWise is a layered Python backend with a React frontend. Lower layers never import higher ones, and all business rules live in one place (`engine/`).

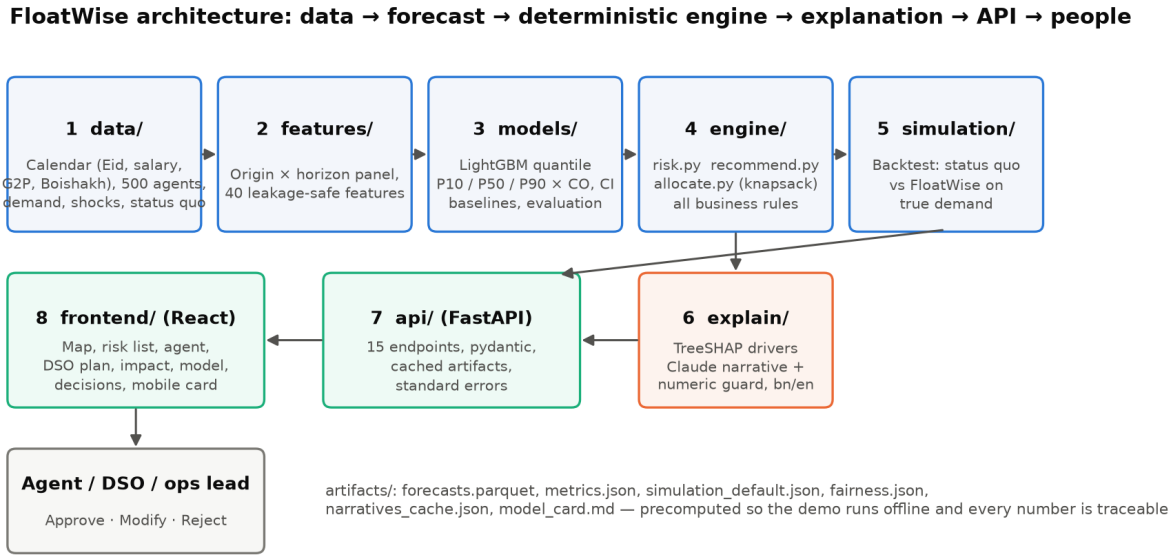


Figure 5. *Layers and data flow.*

4.1 Layers

Layer	Responsibility	Key modules
data/	Bangladesh calendar, 500 agents, true demand, shocks, censored ledger under today's rule	calendar_bd.py, agents.py, demand.py, shocks.py, simulate_status_quo.py, generate.py
features/	Origin × horizon panel with 40 leakage-safe features	build.py
models/	Baselines, LightGBM quantile training, evaluation, batch prediction	baselines.py, train.py, evaluate.py, predict.py
engine/	Stockout probability, recommendation, DSO allocation (all business rules)	risk.py, recommend.py, allocate.py
simulation/	Day-by-day backtest of status quo vs FloatWise on true demand	backtest.py
explain/	TreeSHAP drivers, Bangla/English templates, Claude narrative with guardrail	shap_explain.py, templates.py, llm.py, labels.py
api/	FastAPI service over precomputed artifacts; also serves the built web app	main.py, store.py
frontend/	React + TypeScript single-page app with offline fixtures	pages/, components/, api/, i18n/, fixtures/

4.2 Technology stack

Area	Technology
Language and runtime	Python 3.11+, Node.js 20+
Data and ML	pandas 3.0, NumPy 2.4, PyArrow (Parquet), LightGBM 4.7 (quantile

Area	Technology
	objective), scikit-learn 1.9
Explainability and LLM	LightGBM TreeSHAP (pred_contrib); Anthropic Claude SDK (model from LLM_MODEL env var)
API	FastAPI 0.142, pydantic 2.13, Uvicorn
Frontend	React 19, TypeScript 5.9 (strict), Vite 8, Tailwind CSS 4, Recharts 3, react-leaflet 5 with an offline Bangladesh outline
Quality	pytest (89 tests), ruff, tsc, eslint, Playwright browser walk-through, GitHub Actions (Linux + Windows)
Tooling	run.py cross-platform launcher, Makefile, config.yaml for every assumption

4.3 Data contracts

The pydantic schemas in `backend/floatwise/schemas.py` are mirrored one-to-one in `frontend/src/api/types.ts`. Any contract change updates the SRS, both schema files and the frontend fixtures in the same commit. Money is always an integer number of taka.

4.4 Precomputation

Forecasts for the test period are generated once into `artifacts/forecasts.parquet`. The API reads them, so every GET responds in under 300 ms and the demo runs without internet. The artifacts are deterministic: rebuilding from the same seed reproduces model `fw-c078f4bb` and every number byte for byte.

5. Synthetic data

No real customer, agent or transaction data is used. The generator is deterministic from a seed in `config.yaml`, and every assumption is documented in `docs/DATA_ASSUMPTIONS.md`.

5.1 Scale and split

Item	Value
Period	1 January 2024 → 30 June 2026 (daily)
Agents	500 across 15 districts, about 25 per DSO, 5 DSOs per distributor
Train / validation / test	≤ 30 June 2025 / 1 July – 31 December 2025 / 1 January – 30 June 2026
Eid-ul-Fitr	11 April 2024, 31 March 2025, 20 March 2026
Eid-ul-Adha	17 June 2024, 7 June 2025, 27 May 2026

5.2 Generation model

True daily demand is a product of interpretable multipliers:

> `demand_CO(agent, d) = base[archetype] × size[agent] × weekday × salary(day index) × eid(days to/after Eid) × ramadan × boishakh × g2p × shock(district, d) × trend(d) × lognormal noise`
>
> Cash-in follows the same pattern with its own multipliers. The ledger then plays today's rule of thumb against this demand; on stockout days the observed volume is **censored** (lower than true demand), just as in real data.

5.3 Agent archetypes

Archetype	Base cash-out / day	Base cash-in / day	Signature
urban_residential (125)	Tk 1,50,000	Tk 1,80,000	Cash-in spike before Eid → e-float stockouts
garment_belt (100)	Tk 2,50,000	Tk 1,00,000	Cash-out spike on salary days 1–10 and the Eid bonus
rural_remittance (150)	Tk 80,000	Tk 40,000	Cash-out ×2–3 before Eid as remittance arrives; G2P days
market_hub (75)	Tk 3,00,000	Tk 3,00,000	High volume both ways; Boishakh and pre-Eid shopping
university_area (50)	Tk 60,000	Tk 50,000	Month-start cash-in from families; vacations

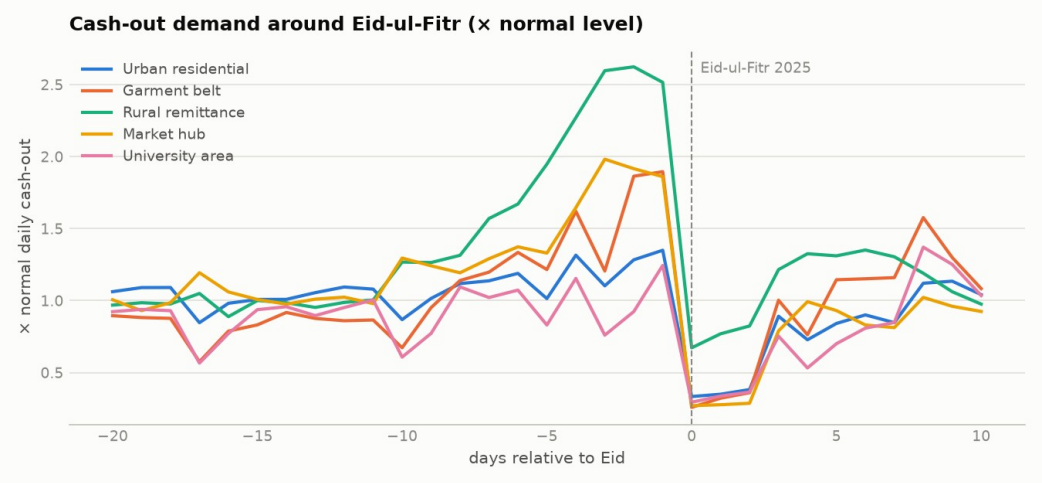


Figure 6. *Eid spike by area type in the generated data.*

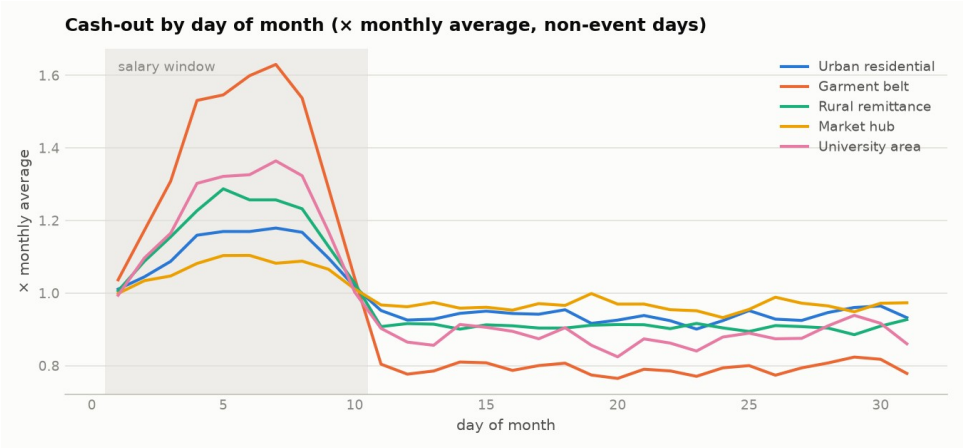


Figure 7. *Salary-day cycle for garment-belt agents.*

5.4 Alignment with upay

upay reality (public)	In FloatWise
Agent cash-out fee by wallet: primary 1.4%, salary 1.0%, remittance 1.0%, disbursement 0.7%	Fee per area type by wallet mix (garment → salary wallet, rural → remittance/G2P blend)
Agent commission 0.41%, distributor 0.10% per cash-in / cash-out	Commission lost on failed transactions; upay keeps fee − 0.51%
Salary disbursement, inward remittance, G2P allowances	Garment salary days, pre-Eid remittance surge, quarterly allowance days
Distributor → DSO → agent network, ~1,54,000 agents	500 agents, 15 districts; impact scaled to 1,54,000
Customer cash-out limit Tk 25,000 per transaction	Synthetic ticket sizes Tk 1,500–3,000

5.5 Data quality checks

`tests/test_data_quality.py` checks that the same seed gives identical files, that cash + e-float equals the agent's float every day, that no balance goes negative, that served volume never exceeds true demand, that stockouts concentrate on event days, and that the Eid, salary and G2P signatures appear where expected.

6. Forecasting model

6.1 Targets and models

- Two targets (cash-out, cash-in) × three quantiles (P10, P50, P90) = six global **LightGBM** models with the quantile objective.
- Direct multi-horizon: the horizon h = 1..7 is a feature, so one model covers the whole week.
- The target is the ratio of demand to the agent's trailing 28-day mean, so one model learns multipliers across agents of very different size.
- Crossing quantiles are re-ordered (rearrangement) so that P10 ≤ P50 ≤ P90 always holds.

6.2 Features (known at the forecast origin)

Group	Features
Horizon	h
Target-date calendar	weekday, Friday, days to Eid-ul-Fitr / Eid-ul-Adha, days after Eid, Ramadan, salary-day index and window, Boishakh, holiday, G2P day index
Lags (≤ origin)	last value, lags 1/6/13, rolling mean/std 7/14/28, rolling max 28
Event memory	agent's multiplier around the previous Eid
Agent static	archetype, float capacity, district, DSO density
Censoring	stockout count in the last 7 days

No leakage: a feature for origin t may use data up to t only; calendar features of the target date are allowed because they are known in advance. `tests/test_no_leakage.py` enforces this by perturbing future data and checking that features do not change.

6.3 Baselines and acceptance

Every model is compared with two baselines: **same day last week** (seasonal naive) and the **trailing 7-day mean** (today's rule). All four acceptance checks pass.

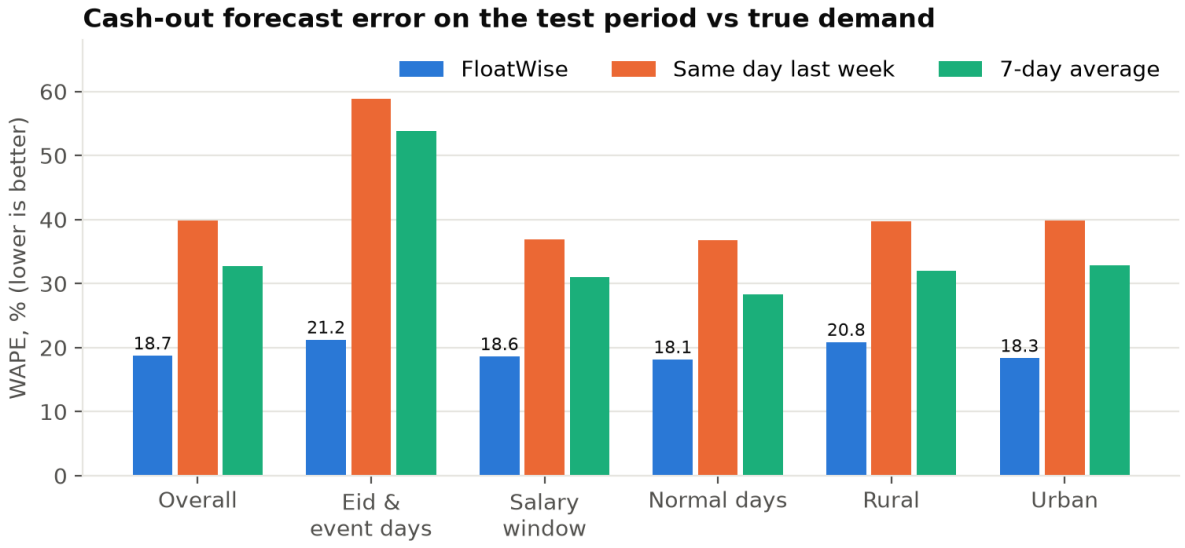


Figure 8. *Cash-out WAPE on the test period, measured against true (uncensored) demand.*

Acceptance check (SRS §6.4)	Target	Result
Overall improvement vs same day last week	≥ 20%	53% (18.7% vs 39.8%)
Event-day improvement	≥ 35%	64% (21.2% vs 58.9%)
P10–P90 coverage	72–88%	75% overall (60% on event days, reported as a limit)
No archetype worse than the baseline	all five	all five better

7. Decision engine

The engine is deterministic, unit-tested code. It turns quantile forecasts and balances into actions and amounts.

7.1 Stockout probability

A piecewise-linear distribution is built from P10, P50 and P90 with explicit tails. The effective opening cash is `open_cash + α × CI_P50` (α = 0.5 for same-day inflows). Its position in the cash-out distribution gives `p_cash_stockout`; the e-float side is symmetric. Risk bands: high ≥ 0.5, medium ≥ θ, else low.

7.2 Recommendation

> `target_cash(s) = ceil_1000( CO_q(s) − α·CI_P50 + Tk 5,000 buffer )`, and symmetrically for e-float.
>
> If cash and e-float targets fit within the agent's total float: top up the risky side to its target.
>
> If they do not fit: apply the best pro-rata split of the float.
>
> If even the P90-level needs exceed the float: **NEEDS_EXTRA_FLOAT**, always escalated to a person.
>
> Otherwise: NO_ACTION. Defaults: service level s = 0.99 and risk threshold θ = 0.10, chosen on validation.

Each recommendation carries reason codes from the calendar and balances (for example `EID_FITR_IN_5_DAYS`, `RAMADAN`, `SALARY_WINDOW`, `CASH_BELOW_NEED`, `RECENT_STOCKOUTS`) and a 7-day outlook of risk and need.

7.3 DSO allocation

Candidates are a DSO's agents with a top-up action. Priority is the **expected shortfall in taka**, computed exactly from the forecast distribution. An exact knapsack optimiser picks the visits that cover the most taka at risk within 10 visits and a Tk 10 lakh cash bag; a greedy planner is kept for comparison. Agents not visited receive the advice in the app, and the backtest assumes 70% of them follow it.

7.4 Business values

Value	Setting
Cash-out fee	1.0–1.4% by area type (upay published tariffs by wallet)
Channel commission	agent 0.41% + distributor 0.10% = 0.51%
Customer churn after a failed cash-out	5% (assumption, labelled in the UI)
Opportunity cost of idle cash	10% a year (assumption)
Network scale-up	~1,54,000 upay agents

8. Explainability and the language model

- **Drivers:** LightGBM TreeSHAP contributions of the median model, converted to taka and shown as the top five drivers with plain labels in both languages.
- **Narrative:** a deterministic Bangla or English template always exists. With an API key, Claude writes a friendlier explanation from structured JSON facts only (no free text from users).
- **Numeric guardrail:** every number in the LLM's text must appear in the facts; otherwise, on refusal, timeout or error, the template is used. The explanation endpoint never fails.
- **Caching:** narratives for the demo are pre-generated into `artifacts/narratives_cache.json`, so the demo works offline.
- **Labelling:** the UI marks the text as AI-generated or template, and keeps predictions, assumptions and generated text visually distinct.

> **The LLM never decides.** Amounts, risk and actions come only from `engine/`. The model explains them in the agent's language.

9. API and user interface

9.1 REST API (FastAPI, prefix /api/v1)

Method	Endpoint	Purpose
GET	/health	Status, model version, data range, demo date, LLM availability
GET	/agents, /agents/{id}	Agent list with risk and action; profile, balances and 28-day history
GET	/agents/{id}/forecast	7-day cash-out / cash-in quantiles with actuals when available
GET	/agents/{id}/recommendation	Action, amount, probabilities, reason codes, 7-day outlook
GET	/agents/{id}/explanation	SHAP drivers and Bangla/English narrative
GET	/dsos, /dso/{id}/plan	DSO list; visit plan under max_visits and max_cash
GET	/network/summary	Risk bands, actions, districts, upcoming events
POST	/simulate	Backtest with custom policy parameters
GET	/metrics/model, /calendar	Evaluation, fairness, tuning, assumptions; events
POST / GET	/decisions, /decisions/summary	Approve / modify / reject log and feedback summary

Inputs are validated with pydantic, errors use one standard JSON format without stack traces, and CORS is restricted to the frontend origin.

9.2 Screens

Screen	What it shows
Network	Eid banner, KPI tiles, risk map of 500 agents, districts by risk
Risk list	Filterable list of agents by district, area type and risk
Agent detail	Recommendation, 7-day outlook, forecast vs true demand, drivers, explanation, approve/modify/reject
DSO plan	Visits ranked by taka at risk within the bag; excluded agents and why
Impact	Backtest, upay fee revenue, network scale-up, live re-run with the service-level slider
Model & fairness	Acceptance checks, error by slice, fairness table, policy tuning, assumptions
Decisions	Decision log and feedback summary
Agent mobile card	One clear Bangla action for the agent

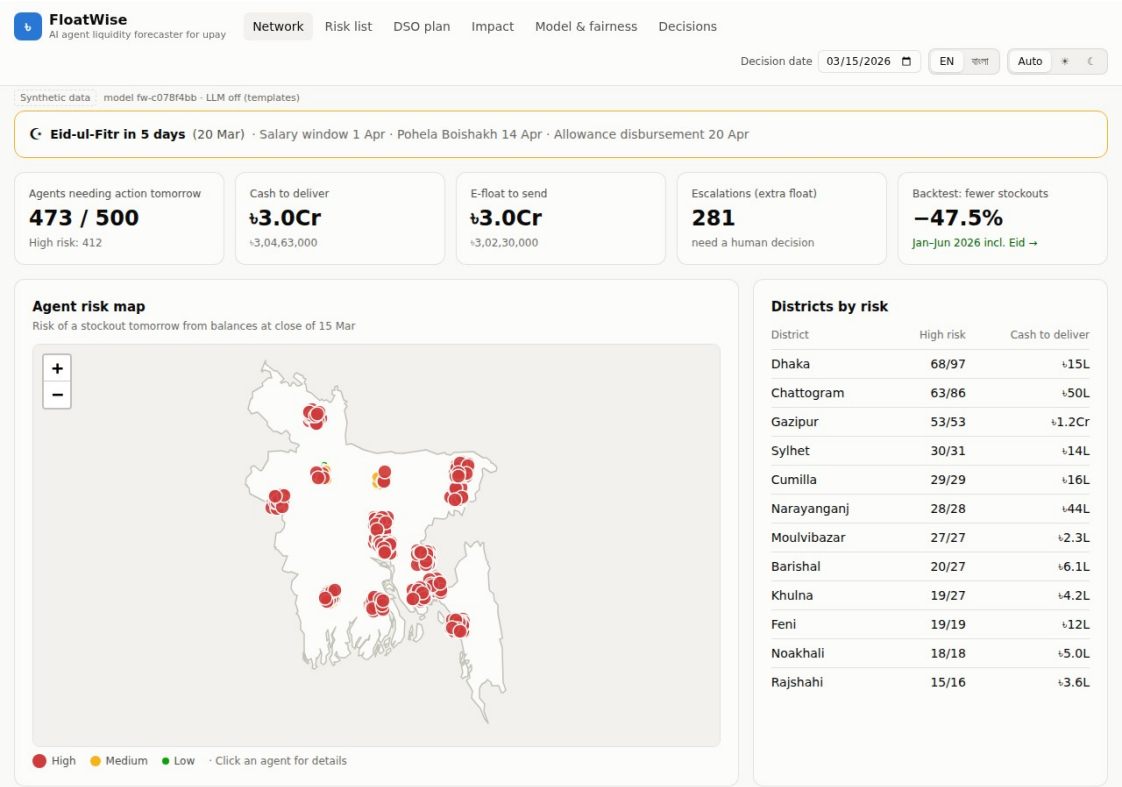


Figure 9. *Network screen on 15 March 2026.*

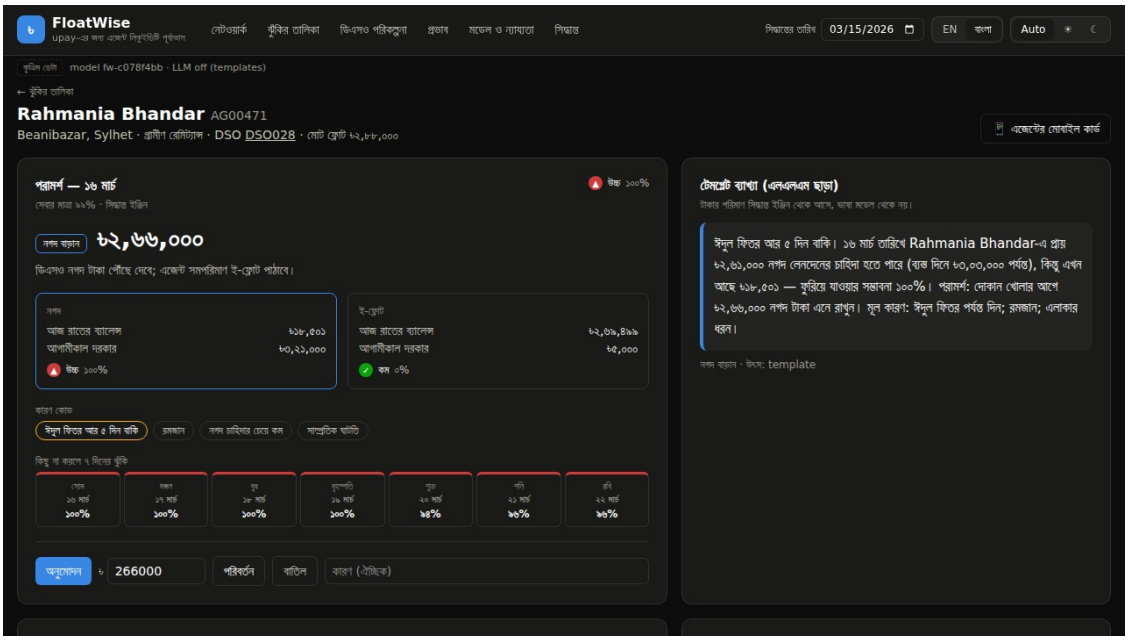


Figure 10. *Agent detail in Bangla (dark mode).*

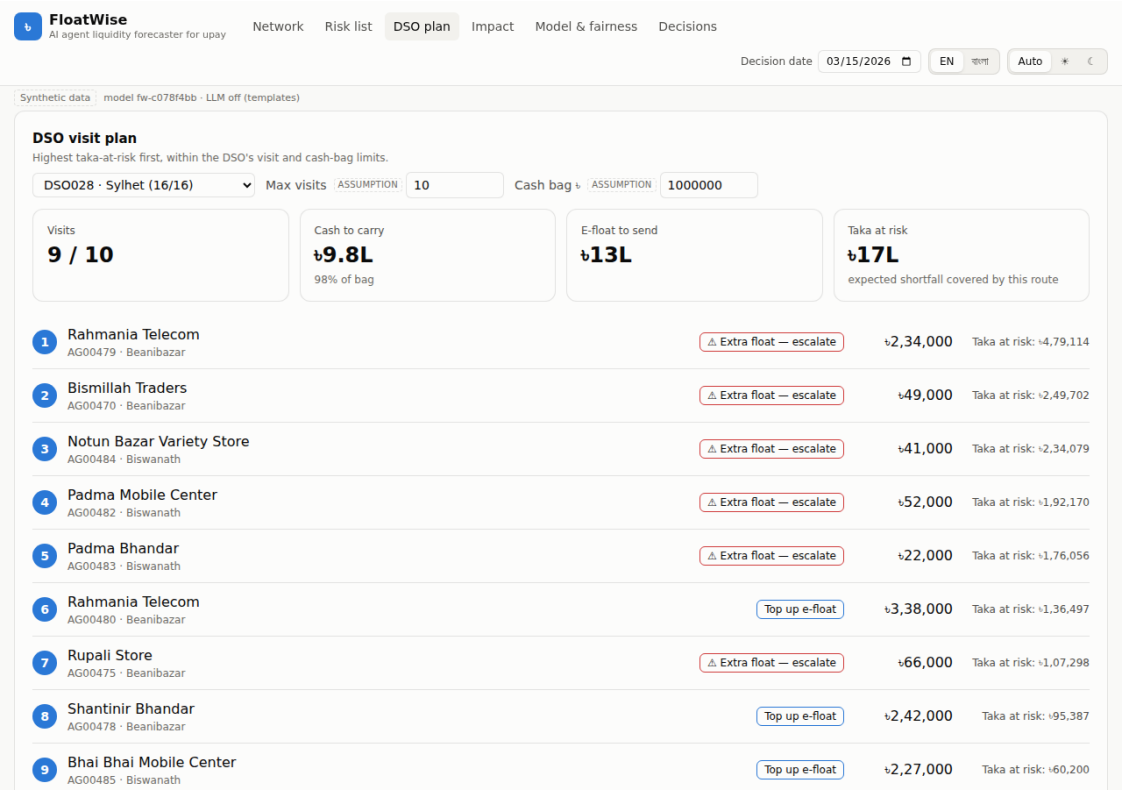


Figure 11. *DSO plan within the visit and cash-bag limits.*

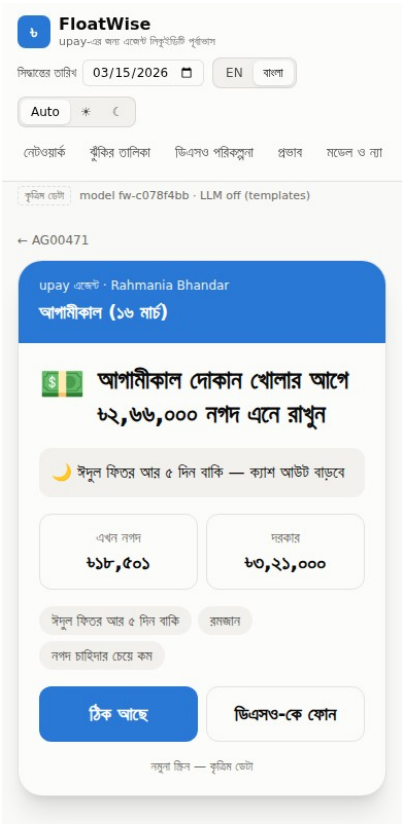


Figure 12. *Agent mobile card in Bangla.*

10. Business impact

The backtest replays January–June 2026 day by day on true demand. Both policies start from the same balances and float; only the timing and size of each top-up differ. The test period was never used for tuning.

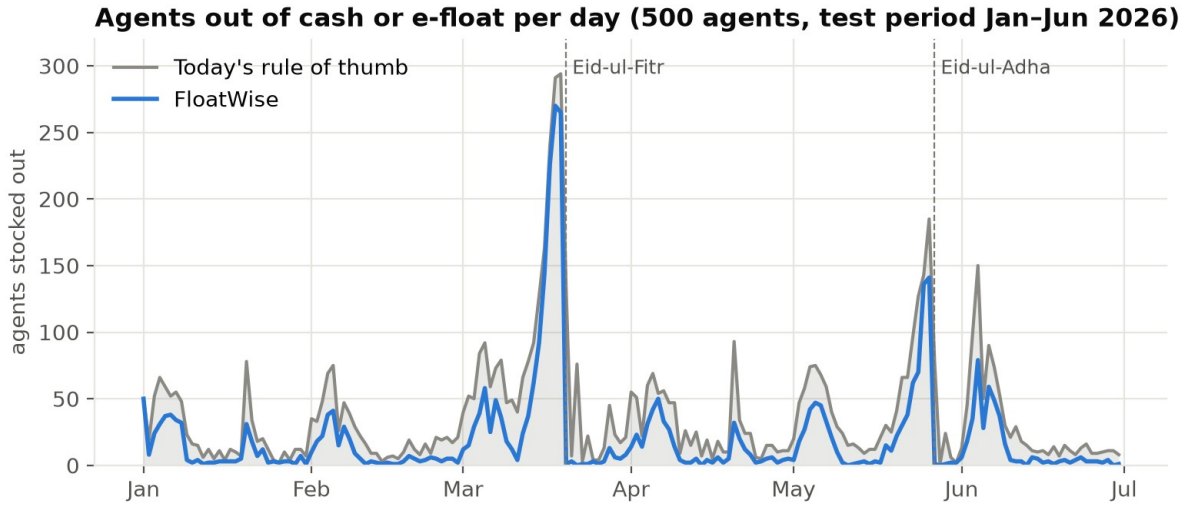


Figure 13. *Agents out of cash or e-float per day.*

Metric (500 agents, 6 months)	Today's rule	FloatWise	Change
Stockout agent-days	6,973	3,664	-47.5%
Failed cash-outs (count)	2,15,101	1,09,349	-49.2%
Failed cash-out value	Tk 52.2 crore	Tk 27.8 crore	-46.7%
upay fee revenue lost	Tk 33.81 lakh	Tk 15.64 lakh	-53.7%
Agent + distributor commission lost	Tk 27.13 lakh	Tk 14.50 lakh	-46.6%
Customers at risk of leaving	10,755	5,467	-49.2%
Share of demand served	98.7%	99.3%	+0.59 pts
Average cash held per agent	Tk 3.53 lakh	Tk 3.61 lakh	+2.3%
DSO rebalancing trips	49,813	51,860	+4.1%

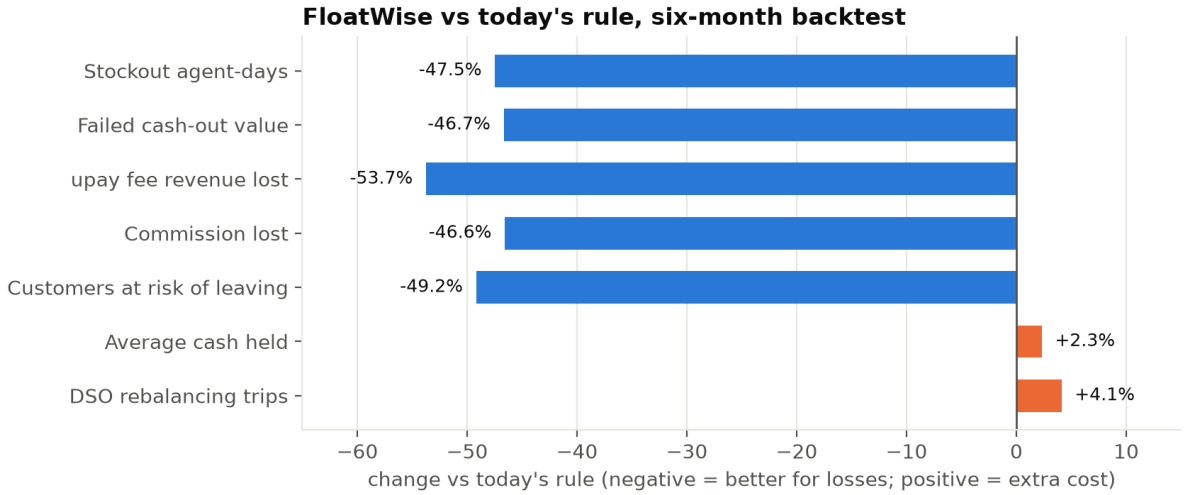


Figure 14. *Change against today's rule: losses fall by about half for a small cost in cash and trips.*

10.1 Unit economics per 1,000 agents per month

Gains	Value	Costs	Value
Stockout-days avoided	1,097	Extra DSO trips	679
Failed transactions avoided	35,742	Extra idle-cash cost (10%/yr)	Tk 67,867
Cash-outs served instead of refused	Tk 8.2 crore		
upay fee revenue recovered	Tk 6.02 lakh		
Commission for agents and distributors	Tk 4.19 lakh		
Customers kept	1,753		

10.2 At upay's network scale

Tk 9.3 Cr
upay fees / month

Tk 6.4 Cr
commission / month

1.69 lakh
stockout-days avoided / month

2.7 lakh
customers kept / month

Per-1,000-agent results scaled to ~1,54,000 agents. Customers kept assumes 5% churn after a failed cash-out.

10.3 One control: the service level

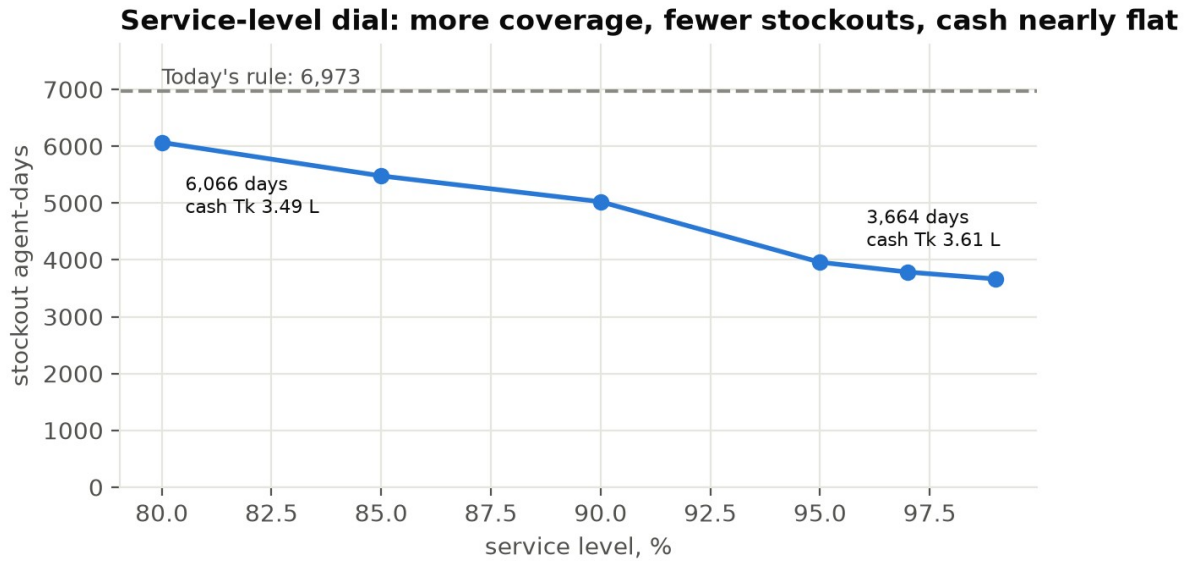


Figure 15. *Service-level sweep (shown on the test period for illustration; 0.99 was chosen on validation).*

10.4 Fairness

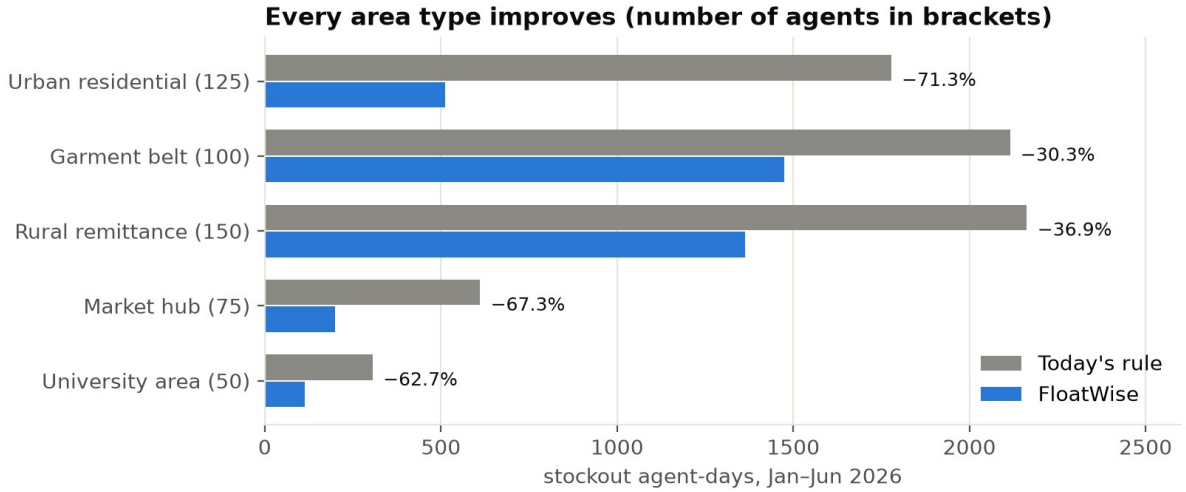


Figure 16. *Stockout agent-days by area type.*

Every segment improves. Rural remittance and garment-belt agents gain least because their total float is small relative to the Eid and salary peaks; those cases are escalated as NEEDS_EXTRA_FLOAT rather than hidden.

11. Design choices made on held-out data

No choice was made on the test period. Each was decided by a script with a written rule, and its evidence is committed.

Choice	Options	Decided on	Selected	Evidence
Stockout-day labels (censoring)	keep / exclude / inflate	Held-out Eid 2025	keep: fewest pre-Eid misses above P90 (57.1% vs 57.4% / 59.5%)	censoring_experiment.json
Quantile architecture	independent / spread	Held-out Eid 2025	independent: spread cut Eid coverage 51% → 42%	quantile_experiment.json
Service level & risk threshold	15-point grid	Validation Jul–Dec 2025	0.99 / 0.10: −55% stockouts, +12.4% trips, +2.4% cash	policy_tuning.json
DSO allocator	greedy / exact knapsack	Validation Jul–Dec 2025	optimal (1,689 vs 1,692 stockout-days)	allocator_comparison.json

12. Responsible AI and security

Principle	How FloatWise implements it
Synthetic data only	No personal data, real names, phone numbers or NIDs anywhere; generic shop names
LLM explains, never decides	Amounts from engine/; LLM gets structured JSON only; numeric post-check with template fallback
Human in the loop	Approve / Modify / Reject on every action; NEEDS_EXTRA_FLOAT always escalates; no automatic money movement
Fairness	Forecast and service checked per area type and rural vs urban; worse segments must be reported
Transparency	AI text labelled; predictions, assumptions and generated text visually distinct; model card published
Honest evaluation	Time-based split; baselines; test period untouched until final evaluation; missed targets reported
Scope	Operational liquidity only; no credit, lending or penalty decisions about agents
Security	Secrets only in .env; pydantic validation; standard errors without stack traces; restricted CORS; works offline

13. Execution: how it was built and how to run it

13.1 Delivery phases

Phase	Delivered
0–1 Contracts	SRS, implementation plan, rules for AI agents, config, schemas, Makefile
2 Data	Calendar, agents, demand, shocks, status-quo ledger; data-quality tests; check plots
3–4 Models	Feature panel with leakage test; baselines; LightGBM quantile models; evaluation and model card
5–6 Engine and simulation	Risk, recommendation, DSO allocation; backtest; policy tuning on validation
7–8 Explain and API	SHAP drivers, templates, Claude narrative with guardrail; FastAPI with 15 endpoints
9 Frontend	Eight screens, Bangla/English, light/dark, offline fixtures
10 Fairness and feedback	Fairness report; decision log and feedback summary
11 Hardening	Offline demo, screenshots, CI on Linux and Windows, fresh-clone rebuild, e2e walk-through, upay alignment

13.2 Pipeline

1. `floatwise.data.generate`: synthetic data into data/generated/.
2. `floatwise.models.train`: six quantile models (plus the baselines).
3. `floatwise.models.evaluate`: test forecasts, metrics.json, fairness.json, model_card.md.
4. `floatwise.simulation.backtest`: simulation_default.json with the service-level sweep.
5. `scripts/plot_data_checks.py` and `scripts/build_narratives.py`: plots and cached explanations.

13.3 Run it (about 6 minutes)

```
git clone https://github.com/redwanhasan980/DIU_AI_Hackathon.git
python run.py setup  install Python and web dependencies (~1–2 min)
python run.py all    data, training, evaluation, backtest (~3 min)
python run.py demo   open http://localhost:8000
```

Needs Python 3.11+ and Node.js 20+. No API key or internet needed once installed.

13.4 Quality assurance

Check	Coverage
89 unit and integration tests	Data quality, no leakage, models, engine, allocator, backtest, explanations and LLM guardrails, API contract and errors
Lint	ruff for Python; tsc and eslint for the web app
Browser walk-through (21 checks)	Map, agent, Bangla, approval, DSO limits, live backtest, fairness, decisions, agent card, dark mode

Check	Coverage
CI	GitHub Actions on every push: backend on Linux and Windows, frontend build
Reproducibility	Fresh clone installs and rebuilds the identical model and numbers with a clean git tree

14. Limitations and next steps

14.1 What FloatWise does not fix yet

Limit	Evidence	Response
Some floats are too small for Eid	265–270 of 500 agents still run out on 18–19 March	Escalate as NEEDS_EXTRA_FLOAT (281 agents flagged on 15 March)
Rural and garment areas gain less	−36.9% and −30.3% vs −62.7% to −71.3% elsewhere	Report per segment; fairness review before rollout
A little more cash and travel	+2.3% cash held, +4.1% DSO trips	Service-level dial trades cash for stockouts
Over-confident at spikes	P10–P90 covers 60% of event days (75% overall)	99% service level chosen on validation
Synthetic data, daily granularity	Shaped on public tariffs and network size; no spill-over to neighbouring agents	Shadow mode on real, anonymised data first

14.2 Path to a pilot

1. **Shadow mode (4–6 weeks):** run nightly on one distributor's anonymised daily agent balances and compare with what DSOs actually did.
2. **Pre-Eid pilot:** that distributor's DSOs use the plan; compare stockouts and failed value with control distributors.
3. **Integrate:** DSO and agent apps (“keep Tk X cash tomorrow”), nightly retraining, drift monitoring per area type.
4. **Scale:** one global model for all ~1,54,000 agents with forecasts precomputed overnight, approvals in an audit log.

> **The ask:** one distributor, anonymised daily agent balances and DSO visit logs, and one Eid season, measured against today's rule.

Appendix A. Glossary

Term	Meaning
Cash-out / cash-in (CO / CI)	Customer withdraws cash from / deposits cash to the agent
E-float	The agent's electronic balance used to serve cash-ins
Stockout	Agent cannot serve a transaction because cash or e-float is exhausted
DSO	Distributor sales officer who visits agents to rebalance cash and e-float
P10 / P50 / P90	Forecast quantiles: 10%, 50% and 90% of outcomes fall below them
WAPE	Weighted absolute percentage error: total absolute error / total actual demand
Service level	Quantile of demand the top-up is sized to cover (0.99 by default)
Censoring	On stockout days the recorded volume understates true demand
G2P	Government-to-person allowance payments

Appendix B. Evidence index

File	Contents
artifacts/metrics.json	Forecast accuracy by slice, coverage, acceptance checks
artifacts/simulation_default.json	Backtest, per-1,000-agent and network-scale value, sweep, segments
artifacts/fairness.json	Per-segment forecast and service checks
artifacts/policy_tuning.json	Validation grid for service level and risk threshold
artifacts/censoring_experiment.json, quantile_experiment.json, allocator_comparison.json	Held-out model and allocator choices
artifacts/model_card.md	Model card
docs/SRS.md, docs/IMPLEMENTATION_PLAN.md, docs/DATA_ASSUMPTIONS.md	Requirements, plan, every assumption with sources